

2026-1st 소프트웨어개론...
PDF

```
mermaid
classDiagram
```

```
%%
```

```
—
```

```
%% 사용자 / 인증
%%
```

```
—
```

```
class User {
```

더 보기 ▾

올려준 클래스 다이어그램을 보니 **횱스크롤 액션 RPG 게임**을 대상으로 꽤 상세하게 설계했네. 수업 자료의 객체지향 설계 원칙(팩토링, 파티셔닝, 계층화) 관점에서 보면 전반적으

로 잘 구성되어 있어.  2026-1st 소프트웨어개론_10_객체지향 설계

좋은 점

1. 책임 분리가 잘 되어 있음

- Player 와 PlayerStats 를 분리
- Inventory 와 Item 을 분리
- GameSession 과 SaveData 를 분리

이런 구조는 응집도를 높이고 유지보수를 쉽게 해줘.

2. 상속 구조가 적절함

```
Item
├─ Equipment
└─ Consumable
```



```
Monster
└─ Boss
```



객체지향의 일반화(상속) 관계를 잘 사용했어.

3. 게임 기능이 충분히 반영됨

- 플레이어
- 몬스터
- 스킬
- 아이템
- 상점
- 랭킹
- 저장 시스템
- 도전과제

까지 포함되어 있어서 기능 요구사항이 잘 드러나.

교수님이 지적할 수 있는 부분

1. Monster도 Stats를 가져야 함

현재

```
Player
└─ PlayerStats
```



만 존재해.

하지만

Monster



도 HP, 공격력, 방어력 등을 갖고 있으므로

CharacterStats



라는 부모 클래스를 만들고

PlayerStats

MonsterStats



로 분리하는 것이 더 객체지향적이야.

2. Player와 Monster의 공통 기능 추출

현재

Player

- attack()
- die()



Monster

- attackPlayer()
- die()

공통 기능이 많아.

예를 들면

Character



라는 추상 클래스를 만들고

Character

- ├ Player
- └ Monster



구조로 바꾸면 좋아.

공통 속성:

hp
attack
defense



공통 메서드:

move()
attack()
die()



3. ItemEffect 클래스가 없음

현재

```
getEffect(): ItemEffect
```



인데

ItemEffect



클래스가 다이어그램에 없어.

UML에서는 반환 타입으로 등장하는 클래스도 보통 정의해 주는 것이 좋아.

4. SaveData의 Inventory 중복

현재

```
Player
└─ Inventory

SaveData
└─ List<Item>
```



인데

```
SaveData
└─ Inventory
```



로 저장하는 것이 더 자연스러워.

5. GameState가 정의되어 있지 않음

현재

```
GameSession
+ GameState state
```



인데

```
GameState
```



클래스 또는 Enum이 없음.

예:

```
GameState
-----
PLAYING
PAUSED
GAME_OVER
CLEAR
```



패키지 다이어그램으로 발전시키기

수업 자료에서는 클래스들을 관련 기능별로 묶어 패키지화하라고 설명하고 있어.

2026-1st 소프트웨어개론_10_객체지향 설계

너의 경우:

```
Authentication Package
├ User
└ Admin

Player Package
├ Player
├ PlayerStats
├ Skill
└ Inventory

Item Package
├ Item
├ Equipment
└ Consumable

Stage Package
```



```
├ Stage
├ Terrain
├ Trap
└ Checkpoint
```

Monster Package

```
├ Monster
└ Boss
```

Shop Package

```
└ Shop
```

Ranking Package

```
├ Score
└ Ranking
```

Session Package

```
├ GameSession
├ SaveData
└ Challenge
```

처럼 묶으면 패키지 다이어그램 과제까지 연결하기 좋아.

총평

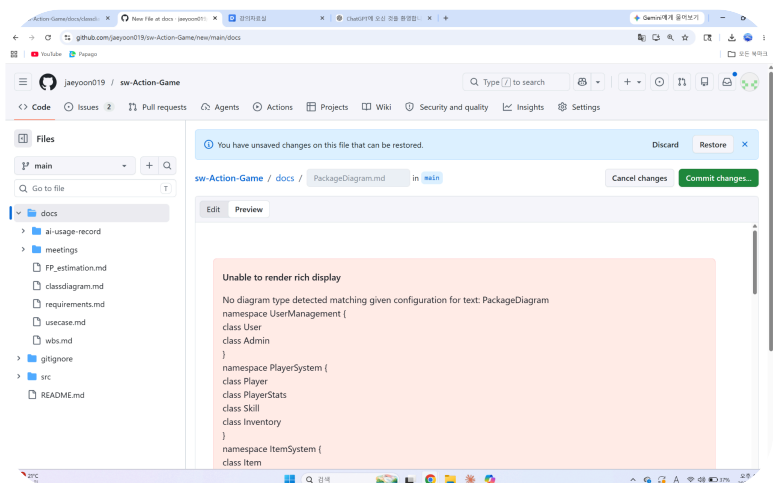
현재 상태만으로도 **80~90점 수준의 클래스 다이어그램**으로 보이고,

- Character 추상 클래스 추가
- MonsterStats 추가
- ItemEffect 정의
- GameState 정의
- 패키지 다이어그램 작성

까지 하면 객체지향 설계 과제에서 상당히 완성도 높은 결과물이 될 것 같아.

과제 요구사항이 "**클래스 다이어그램 → 패키지 다이어그램 → 3계층 아키텍처**"까지 포함된다면, 내가 이 다이어그램을 기반으로 패키지 다이어그램과 3계층(HCI / PD / DM) 구조까지 그려줄 수 있어.

📄 👍 🗨️ ↗️ ↻️ ⋮ 📖 출처



github에서 display할 수 있도록 코드를 만들어줘.

스크린샷을 보니까 GitHub가

No diagram type detected matching given configuration



라고 나오고 있어.

이건 Mermaid 문법의 첫 줄이 빠졌기 때문이야.

현재 너는 아마

PackageDiagram



```
namespace UserManagement {  
...  
}
```

처럼 넣은 것 같아.

GitHub에서 Mermaid를 렌더링하려면 반드시 아래처럼 시작해야 해.

글쓰기



```
```mermaid  
classDiagram

namespace UserManagement {
 class User
 class Admin
}

namespace PlayerSystem {
 class Player
 class PlayerStats
 class Skill
 class Inventory
}

namespace ItemSystem {
 class Item
 class Equipment
 class Consumable
}

namespace StageSystem {
 class Stage
 class Terrain
 class Trap
 class Checkpoint
}

namespace MonsterSystem {
 class Monster
 class Boss
}
```

```
namespace ShopSystem {
 class Shop
}

namespace ScoreSystem {
 class Score
 class Ranking
}

namespace SessionSystem {
 class GameSession
 class SaveData
 class Challenge
}

UserManagement ..> PlayerSystem
PlayerSystem ..> ItemSystem
PlayerSystem ..> StageSystem
PlayerSystem ..> ShopSystem

MonsterSystem ..> ItemSystem
MonsterSystem ..> PlayerSystem

StageSystem ..> MonsterSystem
StageSystem ..> PlayerSystem

ShopSystem ..> ItemSystem

ScoreSystem ..> PlayerSystem
ScoreSystem ..> StageSystem

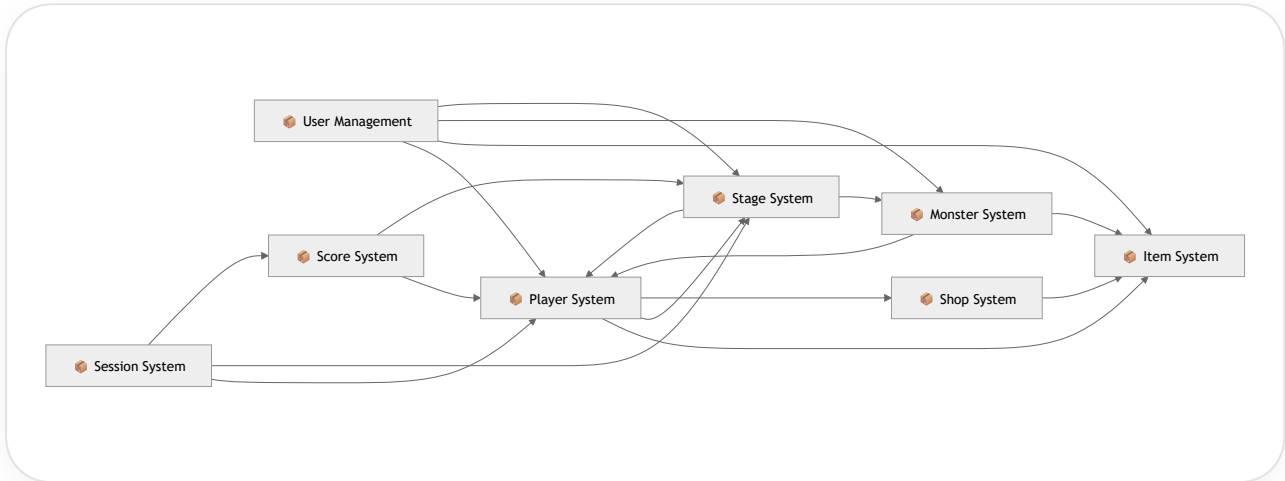
SessionSystem ..> PlayerSystem
SessionSystem ..> StageSystem
SessionSystem ..> ScoreSystem

UserManagement ..> ItemSystem
UserManagement ..> MonsterSystem
UserManagement ..> StageSystem
...
```

그런데 **엄밀히 말하면 이건 패키지 다이어그램이 아니라 "클래스를 namespace로 묶은 클래스 다이어그램" 이야.**

GitHub Mermaid는 UML의 진짜 Package Diagram을 지원하지 않아.

그래서 교수님 제출용이라면 보통 아래 방법을 써.



이 버전이 GitHub에서 **100% 렌더링되고**, 실제 UML 패키지 다이어그램과 가장 비슷하게 보여.

과제 제출용이라면 나는 **flowchart 버전**을 추천해. 교수님들이 보기에다 패키지 간 의존 관계가 더 명확하게 보인다.

📄 👍 🗨️ ↗️ ↻️ ...

flowchart 버전으로 만들어줘.

GitHub Mermaid에서 오류 없이 렌더링되도록 패키지 다이어그램을 flowchart LR 형식으로 작성하면 아래와 같아.

글쓰기



flowchart LR

%% =====

%% Packages

%% =====

UserManagement["🎮 User Management<br/>User<br/>Admin"]

PlayerSystem["🎮 Player System<br/>Player<br/>PlayerStats<br/>Skill<br/>Invent"]

```

ItemSystem["📦 Item System
Item
Equipment
Consumable"]

StageSystem["📦 Stage System
Stage
Terrain
Trap
Checkpoint"]

MonsterSystem["📦 Monster System
Monster
Boss"]

ShopSystem["📦 Shop System
Shop"]

ScoreSystem["📦 Score System
Score
Ranking"]

SessionSystem["📦 Session System
GameSession
SaveData
Challenge"]

%% =====
%% Dependencies
%% =====

UserManagement --> PlayerSystem

PlayerSystem --> ItemSystem
PlayerSystem --> StageSystem
PlayerSystem --> ShopSystem

MonsterSystem --> PlayerSystem
MonsterSystem --> ItemSystem

StageSystem --> MonsterSystem
StageSystem --> PlayerSystem

ShopSystem --> ItemSystem

ScoreSystem --> PlayerSystem
ScoreSystem --> StageSystem

SessionSystem --> PlayerSystem
SessionSystem --> StageSystem
SessionSystem --> ScoreSystem

UserManagement --> ItemSystem
UserManagement --> MonsterSystem
UserManagement --> StageSystem

```